Finite Markov Decision Processes

Richard S. Sutton and Andrew G. Barto, Reinforcement Learning: An Introduction (2nd Edition) Chapter3

유희철

2026-03-28

목차

1. Recap - Chap1
2. Who with Whom?
3. For What?
4. How to measure?
5. How to represent
6. How to solve?
7. The End?

Recap - Chap.1

강화학습 넌 어디서 나타났냐

Recap - Chap.1

강화학습 넌 어디서 나타났냐
갑자기 쏟아지는 Terminology

Reward?

Action?

State?

Value?

Environment?

Policy?

Return?

Episode?

Agent?

Recap - Chap.1

Trial and Error Learning

Optimal Control Theory

Recap - Chap.1

Optimal Control의 문제점 Dynamics가 있어야 해

그럼.. Dynamics를 모르는 환경에서는 이 문제를 못푸네?

모델이 없어도 일단 시행을 하자!

시행착오를 하고 여기서 얻은 결과로 Bellman Equation을 점진적으로 맞춰가자

푸는 방법은 알고 있으니(Optimal Control)

시행착오만 하면 돼 (Trial and Error Learning)

Recap - Chap.1

“Learning from Interaction” 으로 접근하자

근데 제어에 학습을 접목시킬 수가 있나?

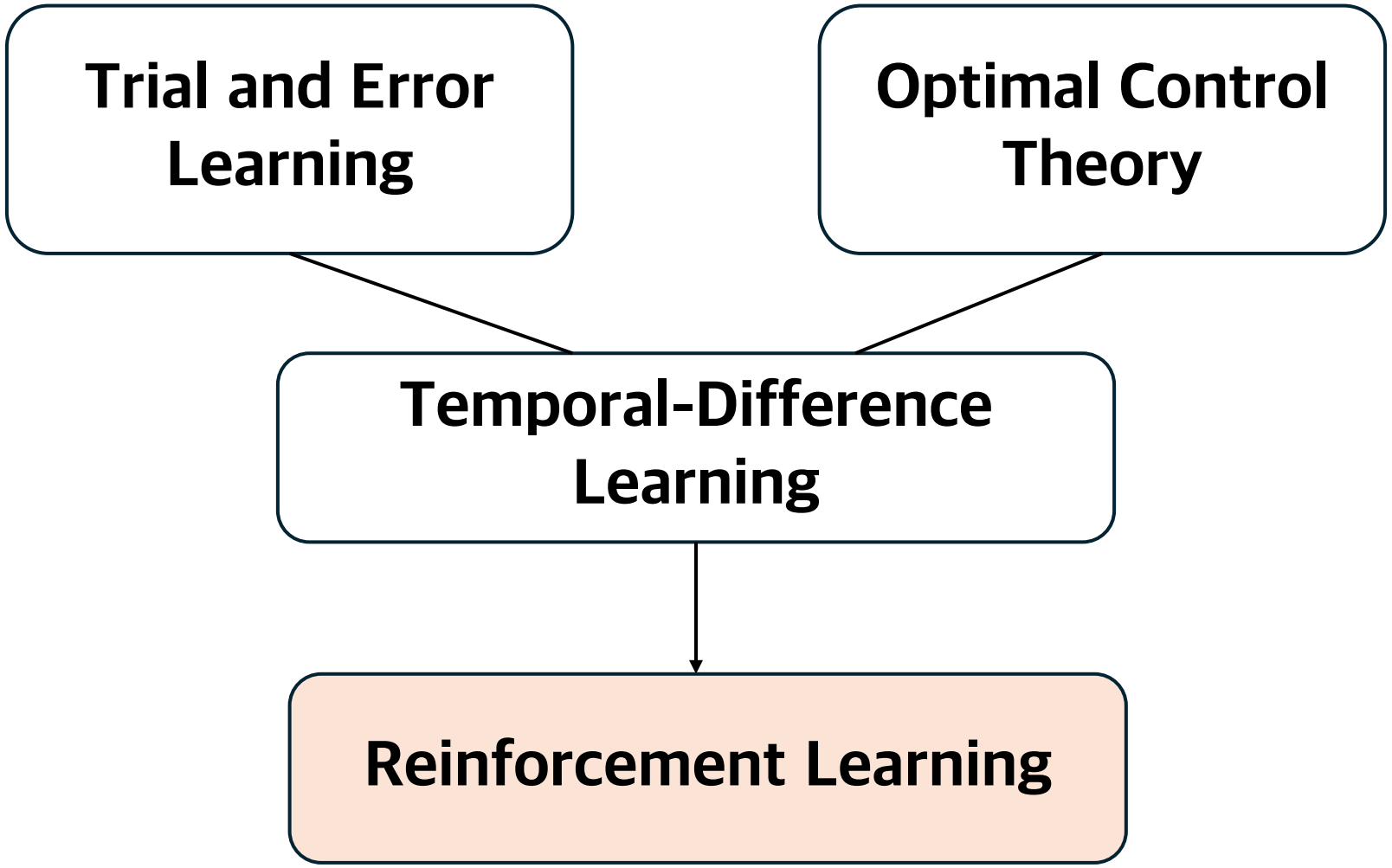
제어는? Offline Backward Computation

학습은? Forward Adaptation

그럼 제어 문제는.. 학습으로 못 푸는거 아니야?

Temporal-Difference Learning

Recap - Chap.1


Trial and Error Learning
Optimal Control Theory
Temporal-Difference Learning
Reinforcement Learning

Recap - Chap.1

강화학습은

Trial and error라는 학습 철학 위에서

Optimal Control이 제공한 최적화 구조를 바탕으로

Temporal Difference Learning 같은 샘플 기반 업데이트를 통해

모델을 모르는 상황에서도 시행착오를 통한 경험으로 근사하여 더 좋은 행동을 학습하는 방식이다.

Recap - Chap.1

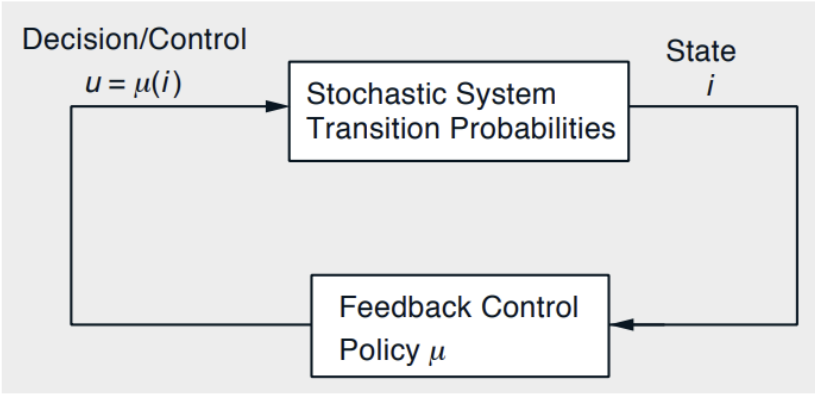


Figure 1.1: Structure of a discrete-time dynamic system under feedback control.

Dimitri P. Bertsekas and John N. Tsitsiklis, *Neuro-Dynamic-Programming* (1996)

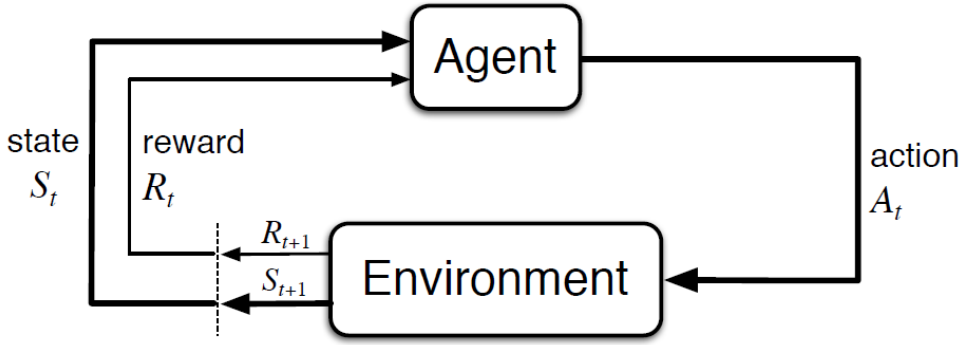


Figure 3.1: The agent–environment interaction in a Markov decision process.

Recap - Chap.1

이제야 합의를 볼 수 있다

Reward?

Action?

State?

Value?

Environment?

Policy?

Return?

Episode?

Agent?

Who with Whom?

The Agent-Environment Interface

Bandit에서는 Stationary State를 가정했었다.

$$q_*(a) = \mathbb{E}[R_t | A_t = a]$$

시간에 따라 각 행동의 보상 분포가 변하지 않는다 + 의사 결정을 나타내는 State Transition이 없다

→ 지금 어떤 상태인지 내 다음 행동이 다음 상태를 어떻게 바꿀지는 하나도 중요하지 않고 실제로 바뀌지도 않음

RL 에서는? 그런 가정은 없음

$$q_*(s, a), v_*(s)$$

지금 상태를 고려해서 보상을 계산해야 함

현재 상태에서의 어떤 행동이 다음 상태에 영향을 준다 → 현재 상태에서의 어떤 행동이 미래의 보상을 바꾼다

Who with Whom?

The Agent-Environment Interface

Assumption : MDP(Markov Decision Process)

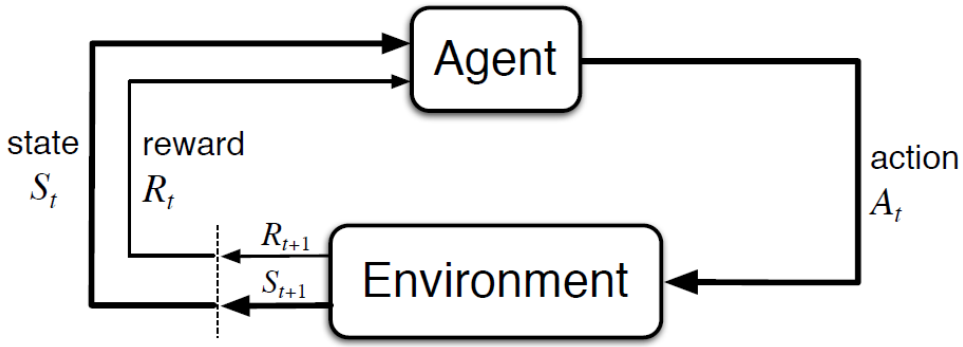


Figure 3.1: The agent–environment interaction in a Markov decision process.

$$R_{t+1}, S_t, A_t, R_t, S_{t-1}, A_{t-1}, R_{t-1}, \dots S_1, A_1, R_1, S_0, A_0$$

$$P(S_{t+1}, R_{t+1} | S_t, A_t, S_{t-1}, A_{t-1}, \dots, S_0, A_0) = \boldsymbol{P(S_{t+1}, R_{t+1} | S_t, A_t)}$$

$$\boldsymbol{p(s', r | s, a)}$$

Who with Whom?

The Agent-Environment Interface

Assumption : MDP(Markov Decision Process)

State는 이전의 모든 Agent-Environment interaction에 대한 정보를 내포하고 있다.

→ Markov Property

$$p(s', r|s, a)$$

$$\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r|s, a) = 1$$

As in all of artificial intelligence, there is a tension between breadth of applicability and mathematical tractability.

Chap 17에서는 non-Markov Observation을 Markov State로 만드는 방법을 다룸

Who with Whom?

The Agent-Environment Interface

Assumption : MDP(Markov Decision Process)

$$p(s', r|s, a)$$

- **State-Transition Probabilities**

$$p(s'|s, a) = \sum_{r \in \mathcal{R}} p(s', r|s, a)$$

- **Expected Rewards for state-action pairs**

$$r(s, a) = \mathbb{E}[R_t|S_{t-1} = s, A_{t-1} = a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r|s, a)$$

- **Expected Rewards for state-action-next-state triples**

$$r(s, a, s') = \mathbb{E}[R_t|S_{t-1} = s, A_{t-1} = a, S_t = s'] = \sum_{r \in \mathcal{R}} r \frac{p(s', r|s, a)}{p(s'|s, a)}$$

Who with Whom?

The Agent-Environment Interface

Assumption : MDP(Markov Decision Process)

General Rule:

- 에이전트(Agent)가 바꿀 수 없는 모든 것은 환경(Environment)로 취급한다
- 그렇다고 환경의 모든 정보를 에이전트가 모른다고 생각하지는 않는다

<u>에이전트와 환경의 경계는 에이전트가 무엇을 아는가가 아니라 무엇을 임의로 통제할 수 없는가에 의해 정해진다</u>

이 경계는 우리가 어떻게 목적을 설계하는지에 따라 달라질 수 있다

→ 어떤 Decision Making Task를 정의하느냐에 따라 달라진다

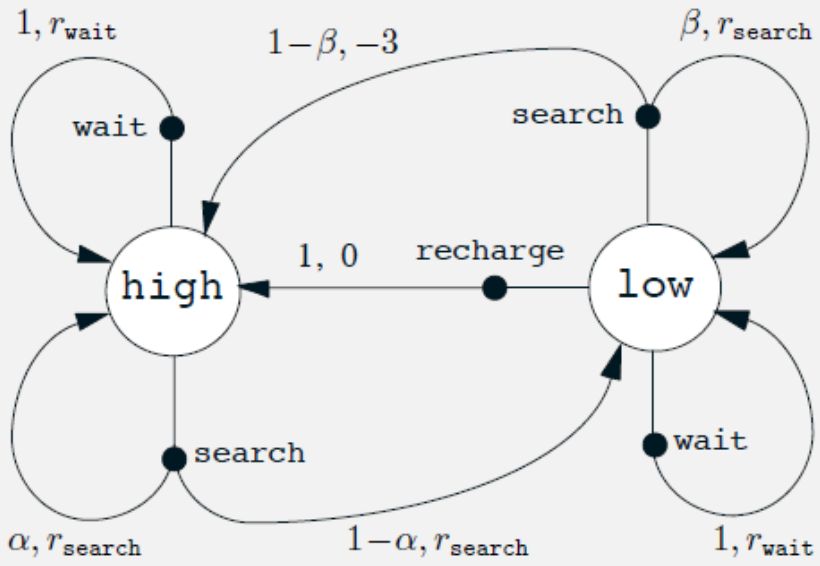
→ 어떤 State, Action, Reward를 정하는지에 따라 달라진다

MDP는 문제를 이 3가지 신호로 축약한 추상화다.

Who with Whom?

The Agent-Environment Interface

s	a	s'	$p(s' \mid s, a)$	$r(s, a, s')$
high	search	high	α	$r_{\texttt{search}}$
high	search	low	$1-\alpha$	$r_{\texttt{search}}$
low	search	high	$1-\beta$	-3
low	search	low	β	$r_{\texttt{search}}$
high	wait	high	1	$r_{\texttt{wait}}$
high	wait	low	0	-
low	wait	high	0	-
low	wait	low	1	$r_{\texttt{wait}}$
low	recharge	high	1	0
low	recharge	low	0	-



For What?

Goals and Rewards

Goal은 Reward로 정의된다 → **Agent의 목적은 시간에 걸친 누적 Reward의 기대값을 최대화 하는 것이다.**

시간 t에 받는 Reward : R_t

Reward Hypothesis:

목표와 목적(Goal)은 Reward라는 스칼라 값 누적합의 기대값을 최대화하는 것이다.

즉각적 보상만을 키우는 것이 아님.

장기 누적 보상을 키우는 것이 중요함 → 장기적으로 원하는 행동을 유도하는 목적 함수

설계할 땐 "어떻게"가 중요한 게 아니라 "무엇을"이 중요

→ 어떻게에 너무 집착하면 Subgoal만 달성하는 사례가 발생할 수도 있음

How to measure?

Returns and Episodes

Goal은 Reward로 정의된다 → Agent의 목적은 시간에 걸친 누적 Reward의 기대값을 최대화 하는 것이다.

시간 t에 한 행동으로 받는 Reward : R_{t+1}

시간에 걸친 **누적 Reward**: $R_{t+1} + R_{t+2} + R_{t+3} + \cdots + R_T$

이를 이제 Return 보상 (G_t)이라 하자

$$\boldsymbol{G_t = R_{t+1} + R_{t+2} + R_{t+3} + \cdots + R_T}$$

근데 모든 문제에 대해 이런 Return을 사용하는게 합리적일까?

예컨대 끝이 없는 문제라면?

How to measure?

Returns and Episodes

Episodic Task

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \cdots + R_T$$

종료 시점이 있는 경우 (Terminal State가 존재하는 경우): 원래의 Return 공식을 사용해도 문제 X

→ **이 한 사이클을 Episode라고 부른다.**

종료 시간 T는 Episode마다 달라질 수 있다

Continuing Task

Terminal State가 없는 경우 계속 돌아가는 공정 제어 같은 예시는 멈춰서는 안된다 $(T \to \infty)$

그럼 Return이 무한대가 될 수도 있네? 그럼 문제가 발생

→ **Discounting Factor γ 를 도입하자**

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$$

How to measure?

Returns and Episodes

Continuing Task

Terminal State가 없는 경우 ($T \to \infty$) Return이 무한대가 될 수도 있음

→ Discounting Factor γ 를 도입하자

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

- Return을 Finite하게 만드는 효과 ($0 \le \gamma < 1$)
- 미래 보상을 현재보다 덜 중요하게 본다
  - k step 뒤의 reward는 γ^k 만큼 줄어든 가치로 취급
- γ값을 조정해서 Agent를 근시안적으로 만들지 원시안적으로 만들지 설계할 수 있다

Return 식은 재귀식

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = R_{t+1} + \gamma G_{t+1}$$

How to measure?

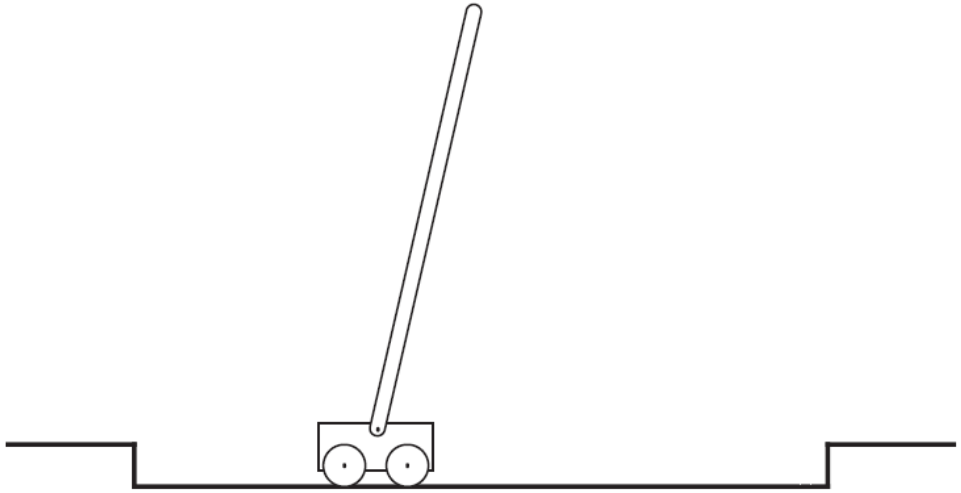
Returns and Episodes

Pole-Balancing Problem: 문제를 설계하는 것은 설계자 마음

Episodic :

실패 전까지 매 Step 보상을 +1

- K step에서 실패하면 보상은 K
- Return을 Maximize 하는 것은 K를 Maximize 하는 것



Continuing :

실패할 때만 보상을 -1, 나머지 시행에서는 보상을 0

- K step에서 실패하면 보상은 $-\gamma^K$
- Return을 Maximize 하는 것은 K를 Maximize 하는 것
    - K가 커져야 $-\gamma^K$가 작아지면서 Maximize 됨

How to measure?

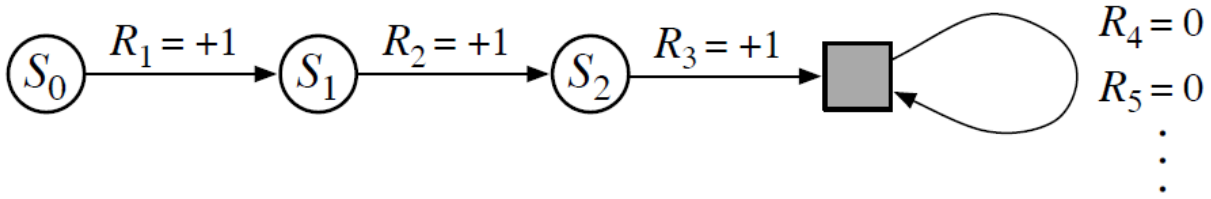
Unified Notation for Episodic and Continuing Tasks

Episodic Task와 Continuing Task의 개념을 하나로 합치자!

그래야 나중에 Bellman Equation과 Value function을 두 개씩 정의하지 않아도 되니까

Terminal State를 Absorbing State로 생각한다

이 State에 진입하면 계속 자기 자신으로 회귀. Reward는 0만 나옴.



$$G_t = \sum_{k=t+1}^{\infty} \gamma^{k-t-1} R_k$$

How to represent

Policies and Value Functions

Policy

각 State에서 Action을 선택할 확률을 주는 함수

$$\pi(a|s)$$

$$\mathbb{E}[R_{t+1}|S_t = s] = \sum_a \pi(a|S_t) \sum_{s',r} rp(s',r|s,a)$$

- 강화학습은 Value Function을 추정함
- 에이전트가 주어진 상태에 있는 것이 “얼마나 좋은지”를 추정하는 것인데
- “얼마나 좋은지”는 미래에 대한 기대 보상으로 정의됨 그리고 이 보상은 에이전트의 행동에 따라 달라짐

→ **이 행동 방식은 “Policy” 정책에 의존하고 따라서 Value function은 정책에 따라 정의가 된다.**

How to represent

Policies and Value Functions

Value Function

Policy π 일 때 State s의 Value Function은 다음과 같이 **정의**된다.

$$v_\pi(s) = \mathbb{E}[G_t | S_t = s] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,|S_t = s\right]$$

- v_π 를 State-Value Function 이라고도 한다

Policy π 일 때 State s에서 Action a의 Value Function은 다음과 같이 **정의**된다.

$$q_\pi(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,|S_t = s, A_t = a\right]$$

- q_π를 Action-Value Function 이라고도 한다

How to represent

Policies and Value Functions

Value Function

Policy $\boldsymbol{\pi}$ 일 때 State s의 Value Function은 다음과 같이 **정의**된다.

$$\boldsymbol{v_\pi(s)} = \mathbb{E}[\boldsymbol{G_t}|\boldsymbol{S_t} = \boldsymbol{s}] = \mathbb{E}_{\boldsymbol{\pi}}\left[\sum_{\boldsymbol{k=0}}^{\infty} \boldsymbol{\gamma^k R_{t+k+1}} \,|\boldsymbol{S_t} = \boldsymbol{s}\right]$$

- 상태 s에서 시작해서 Policy $\boldsymbol{\pi}$ 를 따라가면 얻을 것이라고 기대되는 Return 값

How to represent

Policies and Value Functions

Value Function

Policy π 일 때 State s에서 Action a의 Value Function은 다음과 같이 **정의**된다.

$$q_\pi(s,a) = \mathbb{E}[G_t | S_t = s, A_t = a] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,|S_t = s, A_t = a\right]$$

- 상태 s에서 행동 a를 취하고 Policy π 를 따라가면 얻을 것이라고 기대되는 Return 값

How to represent

Policies and Value Functions

Value Function

가치 함수는 시행착오로 추정한다.

$$\boldsymbol{v_\pi(s) = \mathbb{E}[G_t | S_t = s] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,| S_t = s \right]}$$

$$\boldsymbol{q_\pi(s,a) = \mathbb{E}[G_t | S_t = s, A_t = a] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,| S_t = s, A_t = a \right]}$$

우리는 이 수식에서 모르는 것이 너무 많다. (정책 함수, 상태 전이 확률, Reward의 분포, 궤적의 종류..)

- 직접 시행을 겪으면서 Return을 관측하고 Return을 샘플링해서 Value를 Estimate 함
- Monte Carlo / 큰 수의 법칙
- 결국 진짜 $\boldsymbol{v_\pi(s)}$, $\boldsymbol{q_\pi(s,a)}$로 수렴하게 될 것임

How to represent

Policies and Value Functions

	상태/행동 공간 크기가 작다	상태/행동 공간 크기가 크다
모델을 알고 있다	**Dynamic Programming**	**Approximation Method**
모델을 모른다	**Monte Carlo, TD SARSA, Q-Learning**	**DQN, Actor-Critic, PPO**

How to represent

Policies and Value Functions

	상태/행동 공간 크기가 작다	상태/행동 공간 크기가 크다
모델을 알고 있다	**Dynamic Programming**	**Approximation Method**
모델을 모른다	**Monte Carlo, TD SARSA, Q-Learning**	**DQN, Actor-Critic, PPO**

신경망으로 접근하기도 함

How to represent

Policies and Value Functions

Value Function

각 가치 함수는 재귀적으로 표현할 수 있다

$$v_\pi(s) = \mathbb{E}[G_t|S_t = s] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,|S_t = s\right]$$

$$\begin{aligned} v_\pi(s) &= \mathbb{E}[G_t|S_t = s] = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1}|S_t = s] \\ &= \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)\left[r + \gamma \mathbb{E}[G_{t+1}|S_{t+1} = s']\right] \\ &= \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)\left[r + \gamma v_\pi(s')\right] \end{aligned}$$

이 식을 Bellman Equation for v_π라고 한다.

How to represent

Policies and Value Functions

Value Function

$$v_\pi(s) = \sum_a \pi(a|s)\, q_\pi(s,a)$$

$$q_\pi(s,a) = \sum_{s',r} p(s',r|s,a)\,[r + \gamma v_\pi(s)]$$

q_π에 대해서도 재귀적으로 작성할 수 있다

$$q_\pi(s,a) = \mathbb{E}[G_t|S_t = s, A_t = a] = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1}|S_t = s, A_t = a]$$
$$= \sum_{s',r} p(s',r|s,a)\,[r + \gamma \sum_{a'} \pi(a'|s')\, q_\pi(s',a')]$$

이 식을 Bellman Equation for q_π라고 한다.

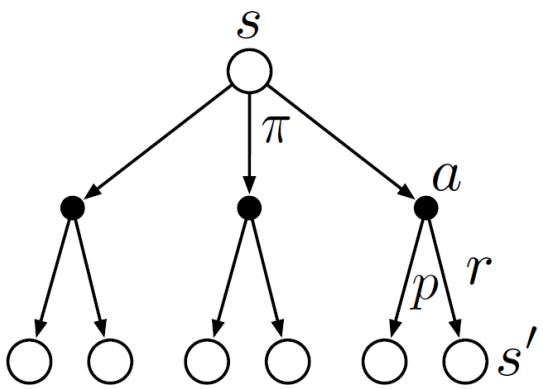
How to represent

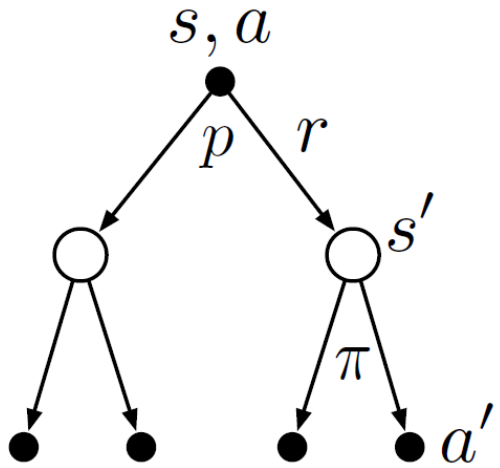
Policies and Value Functions

Backup Diagram

Bellman Equation의 시각화 - 다음 State, Action의 Value 정보를 현재 State / State-Action 으로 가져와서 계산

- 흰색 원 : state
- 검은색 원 : state-action pair





q_π backup diagram

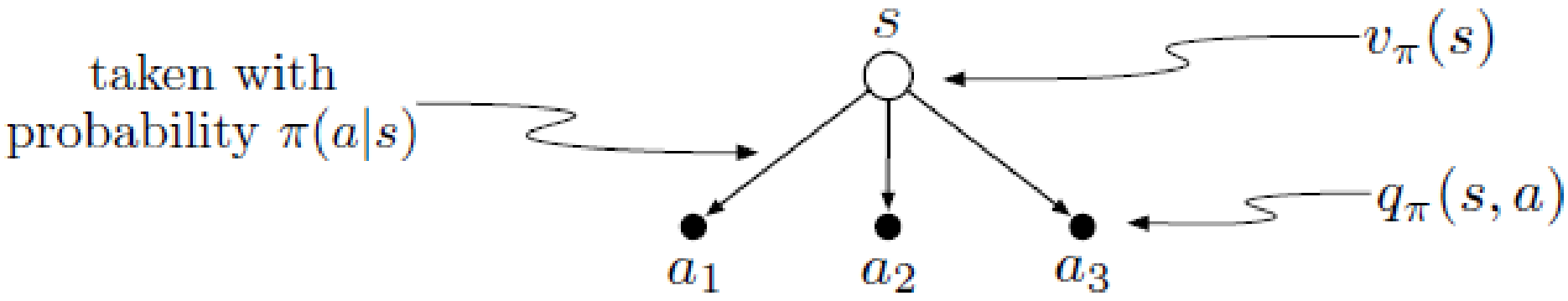
$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)\,[r + \gamma v_\pi(s')]$$

$$q_\pi(s,a) = \sum_{s',r} p(s',r|s,a)\,[r + \gamma \sum_{a'} \pi(a'|s')\, q_\pi(s',a')]$$

How to represent

Policies and Value Functions

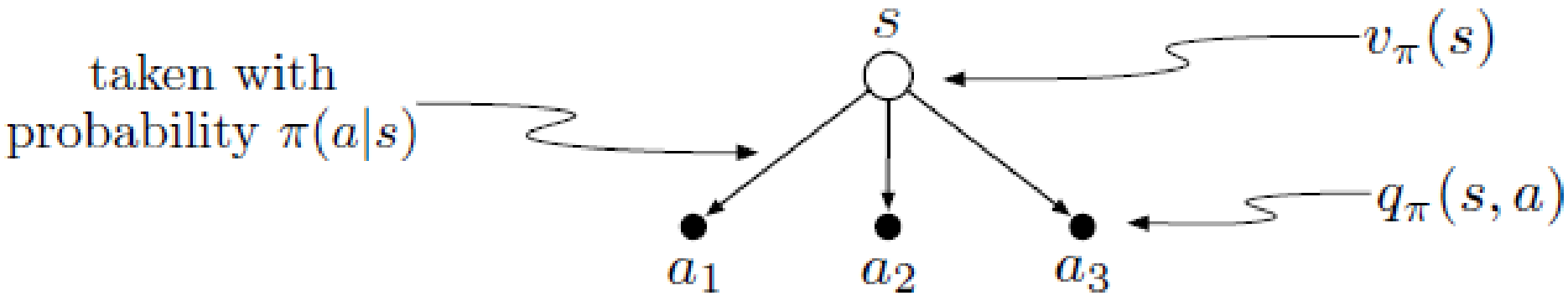
Backup Diagram



How to represent

Policies and Value Functions

Backup Diagram

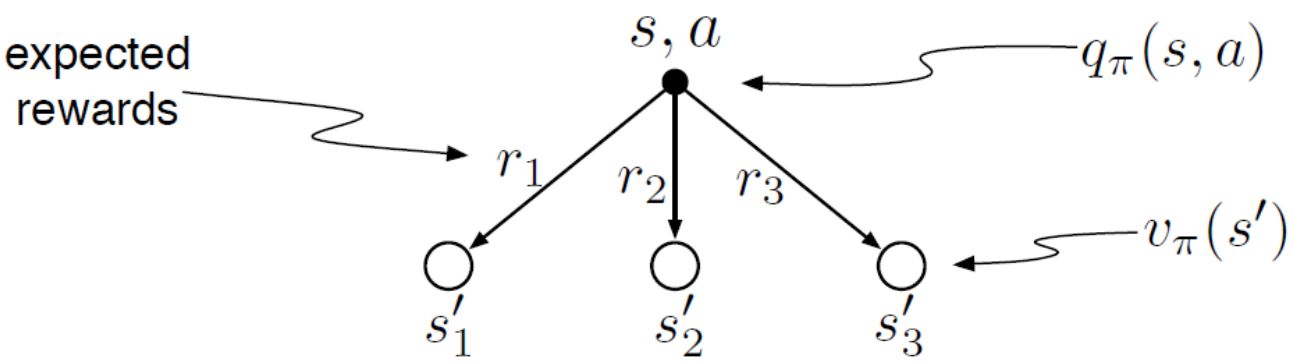


$$v_\pi(s) = \sum_a \pi(a|s)\, q_\pi(s,a)$$

How to represent

Policies and Value Functions

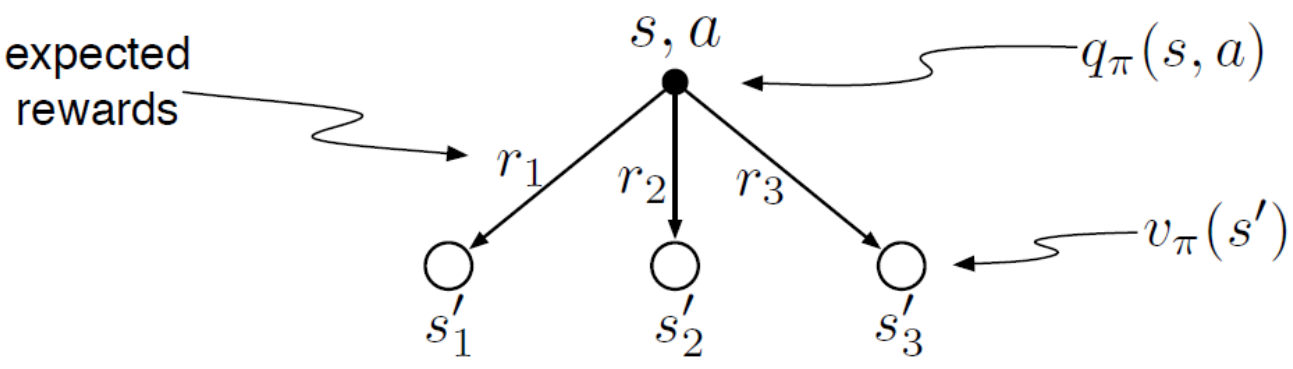
Backup Diagram



How to represent

Policies and Value Functions

Backup Diagram

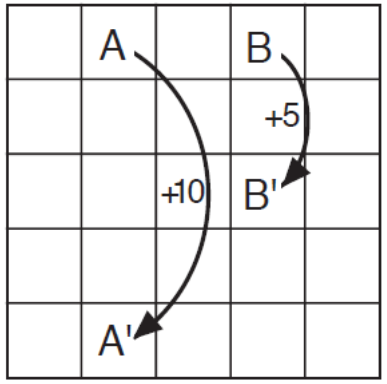


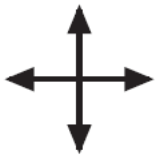
$$q_\pi(s, a) = \sum_a p(s', r|s, a)[r + \gamma\, v_\pi(s')]$$

How to represent

Policies and Value Functions

Example 3.5 Grid - State Value는 어떻게 형성되는가





Actions

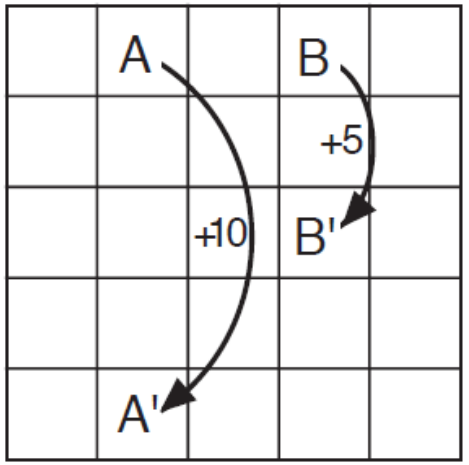
3.3	8.8	4.4	5.3	1.5
1.5	3.0	2.3	1.9	0.5
0.1	0.7	0.7	0.4	-0.4
-1.0	-0.4	-0.4	-0.6	-1.2
-1.9	-1.3	-1.2	-1.4	-2.0

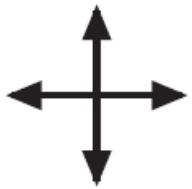
- Action: 상,하,좌,우
- Reward:
  - 그리드 → 그리드 : 0
  - A → A’ : 10
  - B → B’ : 5
  - Off the Grid : -1

How to represent

Policies and Value Functions

Example 3.5 Grid





Actions

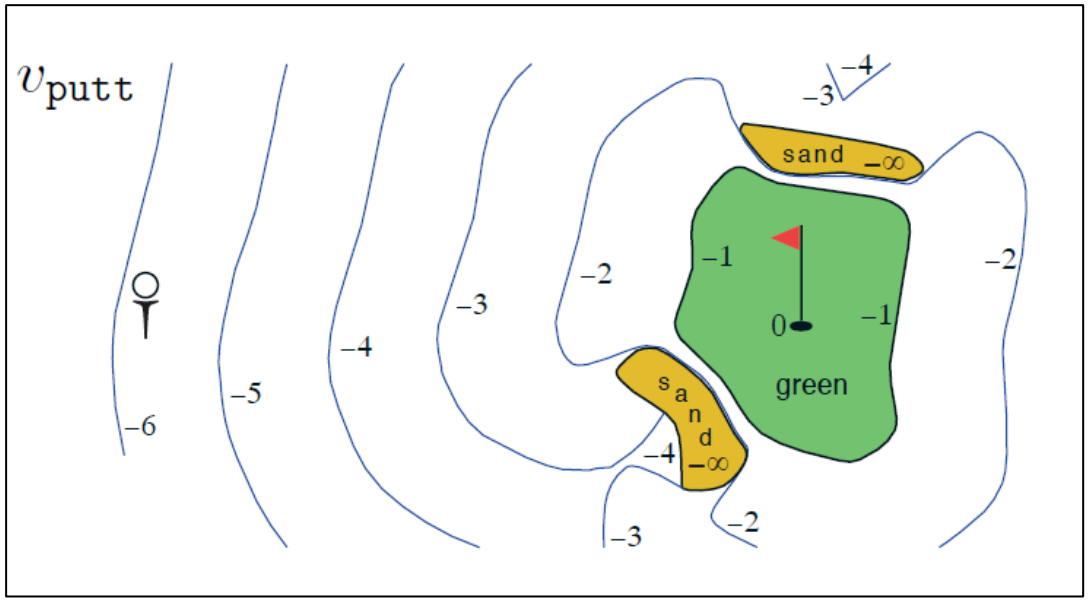
3.3	8.8	4.4	5.3	1.5
1.5	3.0	2.3	1.9	0.5
0.1	0.7	0.7	0.4	-0.4
-1.0	-0.4	-0.4	-0.6	-1.2
-1.9	-1.3	-1.2	-1.4	-2.0

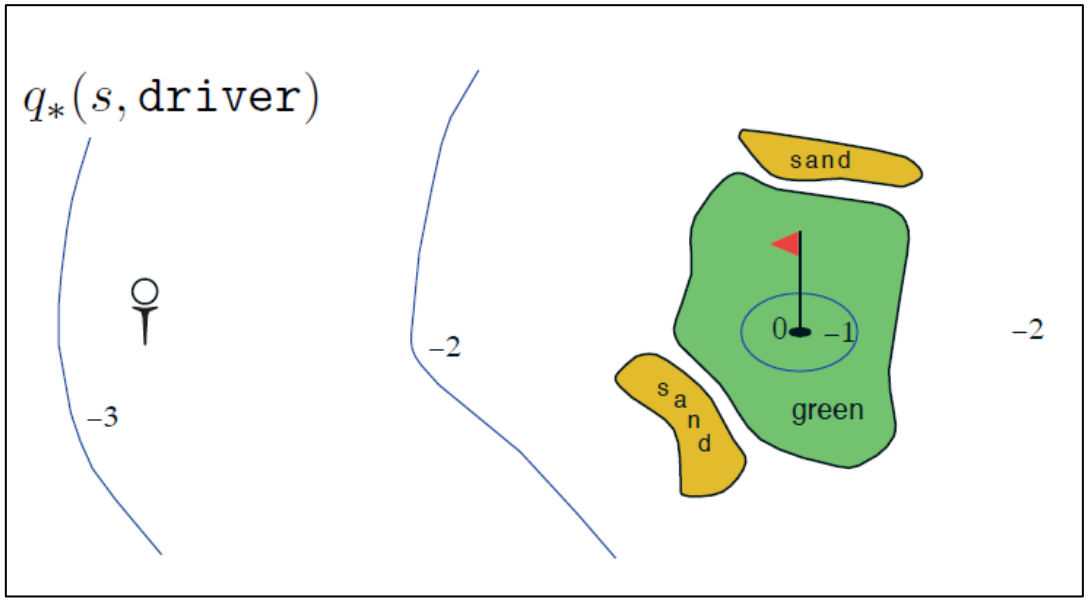
$$v_{center} = \frac{0.9 \times (2.3 + 0.7 - 0.4 + 0.4)}{4} = 0.675 \sim 0.7$$

How to represent

Policies and Value Functions

Example 3.6 Golf





How to solve

Optimal Policies and Optimal Value Functions

모든 것이 정의되었음 : Agent, Environment, State, Action, Reward, Return, Policy, Value Function

이제 강화학습 문제를 풀 일만 남았음

근데 강화학습 문제를 푼다는 건 어떤 의미일까?

→ **장기적으로 최고의 Return을 내게 만드는 Action을 할 수 있는 Policy를 찾는 것이 강화학습 문제를 푸는 것**

→ **제일 좋은 정책이 무엇인가?**

How to solve

Optimal Policies and Optimal Value Functions

Optimal Policy

여러 Policy 중에 모든 state에서 value가 가장 큰 policy를 optimal policy라 한다. 그리고 이를 $\boldsymbol{\pi_*}$ 라고 하자

$$\boldsymbol{\pi \geq \pi' \ \leftrightarrow \ v_\pi(s) \geq v_{\pi'}(s)}$$

$$\boldsymbol{v_*(s) = max_\pi v_\pi(s)}$$

$$\boldsymbol{q_*(s,a) = max_\pi q_\pi(s,a)}$$

이 때 $\boldsymbol{v_*(s)}$를 **optimal state-value function**, $\boldsymbol{q_*(s,a)}$를 **Optimal action-value function** 이라고 한다.

- $\boldsymbol{v_*(s)}$: 상태 s에서 시작했을 때 달성 가능한 최대 Expected Return
- $\boldsymbol{q_*(s,a)}$: 상태 s에서 행동 a를 취했을 때 이후에 달성 가능한 최대 Expected Return

How to solve

Optimal Policies and Optimal Value Functions

Optimal Policy

특징

1. Optimal Policy는 Unique하지 않다
2. Optimal Policy는 모두 같은 optimal state-value function과 같은 optimal action-value function을 공유한다
3. Any MDP에 대해서 Optimal Policy는 Deterministic하다.

$$\boldsymbol{\pi_1 \neq \pi_2 \ \longrightarrow \ v_{\pi_1 *} = v_{\pi_2 *} \ , \ \ q_{\pi_1 *} = q_{\pi_2 *}}$$

How to solve

Optimal Policies and Optimal Value Functions

Optimal Policy

$v_*(s)$와 $q_*(s,a)$ 의 관계 - **Bellman Optimality Equation**

$$q_*(s,a) = \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a] = \sum_{s',r} p(s',r|s,a)[r + \gamma v_*(s')]$$

$$v_*(s) = \max_\pi q_\pi(s,a)$$

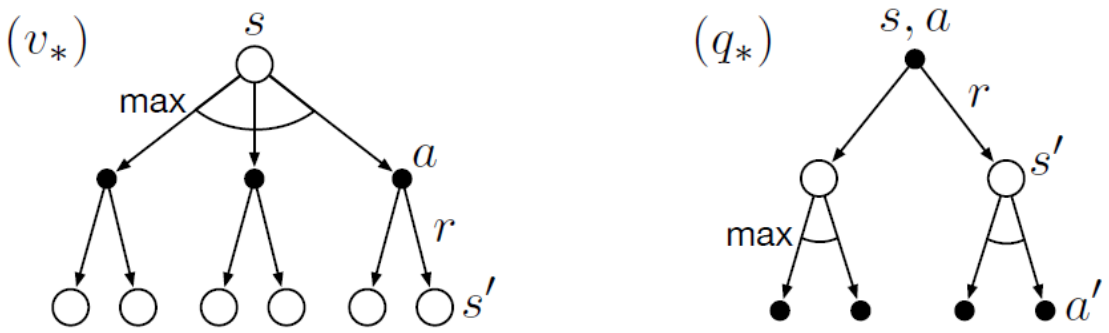


Figure 3.4: Backup diagrams for v_* and q_*

How to solve

Optimal Policies and Optimal Value Functions

Optimal Policy

$v_*(s)$와 $q_*(s,a)$ 를 재귀적으로 표현하자

$$v_*(s) = \max_\pi q_\pi(s,a) = \max_a \mathbb{E}_\pi[G_t | S_t = s, A_t = a]$$
$$= \max_a \mathbb{E}_\pi[R_t + \gamma G_{t+1} | S_t = s, A_t = a]$$
$$= \max_a \mathbb{E}_\pi[R_t + \gamma v_*(S_{t+1}) | S_t = s, A_t = a]$$

$$= \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma v_*(s')]$$

How to solve

Optimal Policies and Optimal Value Functions

Optimal Policy

$v_*(s)$와 $q_*(s,a)$ 를 재귀적으로 표현하자

$$q_*(s,a) = \sum_{s',r} p(s',r|s,a)[r + \gamma \max q_*(s',a')]$$

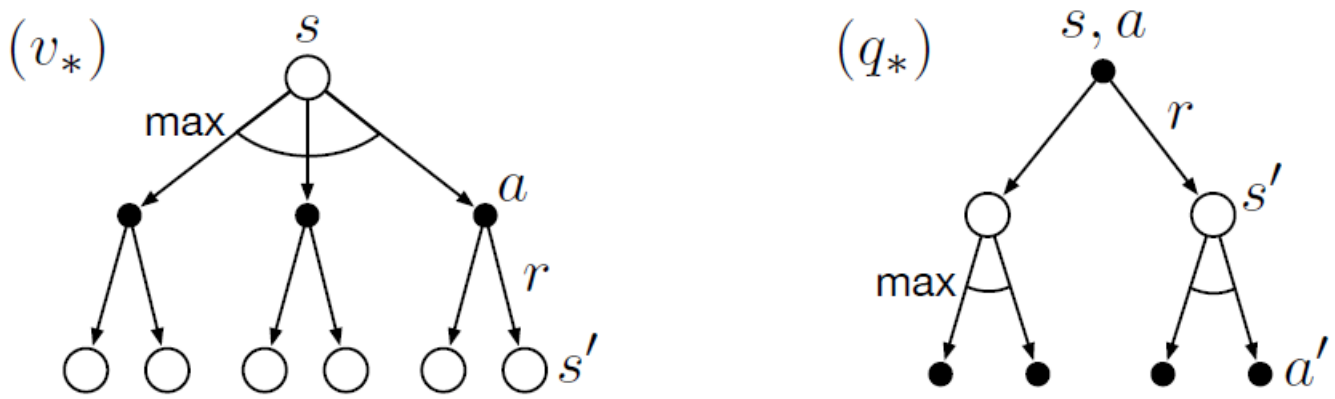


Figure 3.4: Backup diagrams for v_* and q_*

How to solve

Optimal Policies and Optimal Value Functions

Optimal Policy

$$\boldsymbol{a_* = \arg \pi_*(a_*|s) = \text{argmax}\, q_*(s,a)}$$

$$\boldsymbol{a_* = \arg \pi_*(a_*|s)}$$

$$\boldsymbol{= \text{argmax} \sum_{s',r} p(s',r|s,a)[r + \gamma v_*(s')]}$$

- **Prediction** : v_π **구하기**
- **Control** : v_* **찾기**

How to solve

Optimal Policies and Optimal Value Functions

Optimal Policy

왜 $v_*(s), q_*(s,a)$를 알면 Optimal Policy π_*를 알 수 있나?

- $v_*(s)$ **side**

한단계만 진행해도 최적 action을 찾을 수 있다.

Bellman Optimality Equation에서 최대를 달성하는 Action들을 고르면 그 policy는 optimal이다.

→ v_*에 대해 *Greedy* 한 policy는 Optimal이다.

- $q_*(s,a)$ **side**

$v_*(s)$ 보다 편하다 → 아예 action a가 explicit하게 명시되어 있으니까

$v_*(s)$는 적어도 state transition probability를 알고 있어야 함

근데 $q_*(s,a)$ 는 이런 Dynamics를 몰라도 action 선택이 가능함

와 이제 푸는 방법까지 알았네?

- $v_*(s), q_*(s,a)$ 도 배웠고 여기에 장기적 최적 궤적이 압축되어 있다
- 이걸 그냥 Bellman Optimality Equation에 넣어서 풀면 최적 정책 구해지겠구나?

$$a_* = \arg \pi_*(a_*|s)$$

$$= \operatorname{argmax} \sum_{s',r} p(s',r|s,a)[r + \gamma v_*(s')]$$

와 이제 푸는 방법까지 알았네?

- $v_*(s), q_*(s,a)$ 도 배웠고 여기에 장기적 최적 궤적이 압축되어 있다
- 이걸 그냥 Bellman Optimality Equation에 넣어서 풀면 최적 정책 구해지겠구나?

$$a_* = \arg \pi_*(a_*|s)$$

$$= \operatorname{argmax} \sum_{s',r} p(s',r|s,a)[r + \gamma v_*(s')]$$

No

뭐가 문제인데

- **비용**

계산하는데 너무 시간이 많이 걸려요 + 경우의 수가 많으면 계산량이 비례해서 늘어나요

- **너무 큰 상태/행동 공간**

근사한 가치 함수, 정책, 모델을 다루기 위해 큰 메모리가 필요 (Tabular Method로 감당 X)

그리고 사실 에이전트는 항상 최적 행동을 따를 필요가 없다

→ 에이전트가 마주하는 아주 낮은 확률의 어떤 상황에 대한 보상은 전체 보상에 큰 영향이 없기 때문

그래도 기준은 필요하다

Optimality는 에이전트 행동의 방향성을 제시해준다

그리고 Optimality는 알고리즘을 평가하고 설계하기 위한 이론적 토대를 만들어주는 것에 의미가 있다

- Optimal Policy와 Optimal Value Function들은 이론적으로 중요

The End?

Optimality and Approximation

Chap 4 Dynamic Programming

Chap 5 Monte Carlo Methods

Chap 6 Temporal-Difference Learning

...

II. Approximation Solution Methods

...

Q & A